

PiChamber Phase 2: Pi Migration Plan

Status

Approved design plan. This document records the agreed migration from the current OpenCode-backed product to a Pi-native PiChamber. It is a planning artifact only; it does not authorize a partial dual-runtime release or imply that implementation has begun.

This document is the approved plan for the Pi-native migration and is intentionally scoped separately from the completed identity-stabilization work recorded in the changelog.

Purpose

PiChamber began as an MIT-licensed OpenChamber fork. The product direction is now different: PiChamber is to become a high-quality GUI and remote-hosting surface for Pi Coding Agent, while retaining the useful PiChamber-owned product features that do not depend on OpenCode semantics.

The final product must:

- run Pi sessions, tools, models, providers, skills, prompts, and session files through Pi;
- provide desktop and browser UI surfaces backed by a self-hostable PiChamber server;
- support a personal, single-owner workspace server on a local machine or VPS;
- preserve trusted-device access, pairing, direct connections, SSH, tunnels, and relay capabilities;
- remain Pi-native rather than recreating OpenCode concepts that Pi does not have; and
- retain required OpenChamber/OpenCode MIT attribution without retaining an OpenCode runtime dependency or compatibility layer.

Final Architectural Decision

PiChamber will use the ****Pi SDK directly****, inside a PiChamber-owned session-runtime daemon. It will not use OpenCode, a managed OpenCode process, a Pi RPC-worker fleet, or Pi's experimental remote protocol as its product runtime.

```

Browser / Electron renderer / remote trusted device
    |
    | authenticated PiChamber HTTP + WebSocket API
    v
PiChamber web server / Electron in-process backend
- UI assets, auth, trusted devices, pairing, tunnels, relay
- Git, filesystem, terminal, previews, notifications
- API validation, event fan-out, remote-host selection
    |
    | private local IPC only
    | Unix domain socket (macOS/Linux)
    | named pipe (Windows)
    v
PiChamber session-runtime daemon
- imports @earendil-works/pi-coding-agent SDK directly
- creates and owns Pi AgentSessionRuntime instances
- owns session queues, event subscriptions, recovery, and SDK lifecycle

```

```

- uses the normal Pi agent directory and selected server-side cwd
      |
      v
Pi configuration and state on the host
~/.pi/agent/auth.json
~/.pi/agent/settings.json
~/.pi/agent/models.json
~/.pi/agent/sessions/
~/.pi/agent/skills/, prompts/, AGENTS.md, and trusted project resources

```

The browser never connects to the daemon. The daemon is an internal PiChamber runtime service, not a separately installed Pi server and not a public listener.

Why SDK rather than RPC

Pi's SDK is the intended integration surface for Node applications that need a custom UI, direct session state, typed events, custom controls, and direct model/runtime access. It does not require a globally installed 'pi' executable or a background Pi server. It runs the agent in the process that imports it.

The daemon preserves a meaningful boundary between the browser/API product server and the agent runtime without adopting Pi's currently experimental remote protocol. A PiChamber-owned IPC contract can be tailored to PiChamber's durable session and reconnection requirements while preserving direct SDK behavior.

Scope

Initial supported runtime surfaces

The first Pi-native release supports:

1. Electron desktop, including local embedded PiChamber backend operation and remote-instance connections.
2. PiChamber web server, including a browser UI hosted locally or on a VPS.
3. Direct remote access to a PiChamber server on LAN, private-network, DNS, reverse-proxy, or HTTPS addresses.

VS Code, Capacitor native mobile, and hosted mobile are deferred as client surfaces. Their packages should not be broken gratuitously, but they do not block the Pi core release and must not pretend to support Pi sessions before a deliberate later port.

Deployment model

A PiChamber server is a single-owner, single-trust-domain workspace server. It can be self-hosted on a workstation, home server, or VPS. When a desktop client connects to a remote server, Pi tools, terminals, Git, files, previews, sessions, and credentials operate on the remote server's filesystem and user account. PiChamber does not create a desktop-to-VPS filesystem bridge in this phase.

The retained deployment contract is:

- 'pichamber serve' for a headless browser/API server;
- Electron importing the PiChamber backend in-process as it does today;
- Docker and systemd deployment documentation and support;
- direct, SSH-forwarded, private-network, tunnel, and relay connection options;

- a session daemon started, reused, health-checked, and stopped by PiChamber runtime ownership.

Non-goals

The following are deliberately not part of the initial Pi migration:

- OpenCode process spawn, proxy, API client, SDK, config, updater, routes, event types, or compatibility facade.
- MCP management.
- OpenCode plugins, agents, commands, skill catalog copy-installer, and system-prompt optimizer.
- Voice, dictation, text-to-speech, and text-to-speech summarization.
- Goal Mode, pinned/reinjected context, Plan Mode, session assist, auto-review, multi-run/fusion, and PiChamber-managed worktrees.
- Scheduled tasks and loop-file scheduling.
- A PiChamber GUI-extension API or bundled subagent feature in the Pi core milestone.
- A remote-to-local execution bridge.
- Multi-tenant accounts, per-user filesystem isolation, or a public multi-user service model.
- Migration of existing OpenChamber/OpenCode server state, sessions, credentials, or VPS data.
- Pi's experimental remote server/protocol/harness as a PiChamber production dependency.

Product Decisions and Invariants

Pi is authoritative for agent state

PiChamber must use Pi-native state rather than copy or reinterpret it as a competing source of truth:

Concern | Authority

Provider credentials and OAuth tokens | Pi 'auth.json' or provider environment resolution

Custom providers and models | Pi 'models.json'

Global Pi settings | '~/.pi/agent/settings.json'

Project Pi settings | '<project>/.pi/settings.json'

Session transcript/tree/model changes | Pi JSONL sessions

Skills and prompt templates | Pi resource discovery and configured Pi packages/paths

Pi global/project instruction files | Pi 'AGENTS.md' discovery

UI theme, layout, host list, pairing, PiChamber defaults, archive state | PiChamber data root

PiChamber must not copy provider credentials into its own settings. Provider UI can submit a new API key or OAuth result to Pi's credential APIs, but subsequent browser reads expose status only, never stored keys, tokens, or raw credentials.

New-session selection precedence

PiChamber owns product defaults for a newly created session. The intended resolution order is:

1. an explicit model or thinking selection made in the new-session UI;
2. configured PiChamber default model/thinking, when present;
3. Pi settings fallback when the PiChamber setting is unset;
4. Pi's normal available-model fallback.

After a user changes model or thinking inside a session, the Pi session's recorded choice becomes

authoritative for that session. PiChamber must not reset it merely because a global PiChamber default changes later.

Pi has no normal-session equivalent of OpenCode's selectable 'defaultAgent'. PiChamber removes that setting rather than presenting a nonfunctional mapping. A future PiChamber modes/extensions design may add an intentional replacement.

Trust, tools, and extensions

- Preserve Pi project trust for project-local settings, skills, prompts, and extensions. The SDK host must present the trust decision through PiChamber UI rather than silently trusting project resources.
- Follow Pi's no-permission-popup default. PiChamber does not add a mandatory confirmation before every Bash, edit, or write tool call in this phase.
- Native third-party Pi extensions are disabled by default during the core milestone. The setting surface may later make extension enablement configurable, subject to project trust and an explicit allow policy.
- Pi subagents are not a core Pi runtime feature; the documented implementation is an extension that launches separate Pi work. Subagents therefore wait for the extension-support milestone.

Sessions, concurrency, and recovery

- Trusted paired clients have equal control of a shared session.
- One session has one active Pi run at a time. Incoming user input while running is explicitly routed as steering or follow-up work according to PiChamber queue behavior.
- The session daemon publishes authoritative snapshots and sequenced transient events. The client must use a snapshot plus sequence watermark on reconnect, then resume deltas without duplicating or dropping streamed content.
- A browser disconnect or PiChamber web-server restart must not stop daemon-owned work.
- A planned daemon restart should wait for sessions to settle when practical.
- A daemon crash or forced daemon restart interrupts active runs. PiChamber must mark them interrupted, reload persisted Pi sessions, and resync clients. It must never display stale partial output as a successful completion.
- Pi session JSONL is the source of truth for session contents. PiChamber archive state is a separate UI-only sidecar keyed to Pi session identity/path. Archive hides sessions only in PiChamber and never modifies Pi JSONL or Pi CLI listing behavior.

Attachments

The existing data-URL attachment model can put a whole XLSX/PDF/binary payload into model context. PiChamber replaces it with file-path attachments:

1. The browser uploads original bytes to the active server.
2. PiChamber writes the file to OS temporary storage, using a Pi-style path such as '/tmp/pi-clipboard-<uuid>.<extension>'.
3. PiChamber gives the Pi session concise metadata and the server-local file path.
4. Pi tools inspect or manipulate that path through normal tools rather than receiving base64 document content in model context.

PiChamber does not add persistent attachment storage, a PiChamber attachment directory, or a PiChamber cleanup/retention policy in this phase. OS-level temporary-file cleanup is outside PiChamber scope. Resumed sessions must not claim that a historical temporary attachment remains available when the path is gone.

Providers

The Providers UI remains a first-class PiChamber feature, but becomes Pi-native:

- show all providers and models Pi supports, including OpenCode-branded model providers such as Zen/Go when Pi exposes them;
- support existing-provider API-key entry and Pi-supported OAuth/browser/device flows;
- support custom provider/model definitions backed by Pi 'models.json';
- write credentials only through Pi's model runtime/auth storage;
- never return saved credentials to a browser;
- remove OpenCode-specific provider metadata, OAuth proxy behavior, deferred OpenCode restart behavior, and configuration formats.

Skills, prompts, snippets, and magic prompts

- Pi global/project skills and prompt templates are used directly.
- PiChamber Snippets becomes a UI for native Pi prompt-template Markdown files, including global '~/.pi/agent/prompts/' and trusted project '.pi/prompts/' locations.
- PiChamber Magic Prompts remain PiChamber-owned feature configuration.
- The existing OpenChamber catalog is not ported as a copy-based installer. It currently sparse-copies skill directories from sources such as Anthropic and ClawdHub into OpenCode/'agents' locations, is coupled to OpenCode discovery, and has no Pi package semantics.
- A later skills-installation surface may curate packages, but installation/update/remove must use Pi-native package/resource configuration and honor the extension trust policy.

Behavior and instructions

Remove the OpenCode system-prompt optimizer entirely. PiChamber provides an editor/view for Pi global '~/.pi/agent/AGENTS.md' and selected-project applicable 'AGENTS.md', with clear scope labels and project-trust behavior. PiChamber does not introduce a separate system-prompt store.

Feature Disposition

Pi core milestone

Capability | Disposition

Pi session creation, resume, naming, tree navigation, fork/clone, deletion | Keep through Pi-native APIs

Streaming chat, reasoning, tools, abort, steer, follow-up | Keep and rebuild on Pi events

Model and thinking selection | Keep, with PiChamber new-session defaults

Direct remote server connection | Keep

Password, passkeys, trusted devices, pairing | Keep existing single-owner model

Provider authentication/model selection | Keep as Pi-native UI

Project picker/recent server-local directories | Keep

Git UI, filesystem browser/editor, server terminal | Keep in or immediately after the core cutover

Native Pi skills/prompts and PiChamber magic prompts | Keep

Snippets | Reimplement as Pi prompt templates

Archive | Keep as PiChamber-only metadata

Follow-up milestones

Capability | Disposition

Changes walkthrough | Retain; first advanced PiChamber workflow

Small utility model | Retain for walkthrough, Git commit messages, PR descriptions, and project-notes summaries only

GitHub OAuth, issues, pulls, PR workflow | Port after core

Preview proxy | Port after core

Notifications and push delivery | Port after core

Quota display | Port after core using Pi provider state where possible

Desktop mini-chat, tray, deep links, updater refinements | Port after core

Tunnels and E2EE relay | Retain product direction; port after direct connections are stable

Skill package/catalog UI | Design after extension/package policy is mature

Native Pi extension support and subagent workflow | Later, configurable and trust-gated

Custom session folders and retention cleanup | Later; archive ships first

VS Code and mobile Pi clients | Later, intentional runtime ports

Removed for this migration

OpenCode runtime, SDK client, process lifecycle, proxy, config entities, OpenCode API route/event contracts, MCP, OpenCode plugins and agents, OpenCode commands editor, OpenCode skill catalog installer, voice/dictation/TTS, Goal Mode, pinned context, Plan Mode, session assist, auto-review, multi-run, PiChamber-managed worktrees, scheduled tasks, loop files, OpenCode system-prompt optimizer, OpenCode updater, and OpenCode compatibility aliases.

Workstreams

Workstream 0: Record the boundary and prepare the repository

Before executable work:

1. Add the exact Pi SDK dependency only after implementation approval.
2. Pin and review the Pi SDK version deliberately; do not treat 0.x minor changes as transparent updates.
3. Create Pi-owned module boundaries rather than adding Pi behavior to existing 'lib/opencode' modules.
4. Inventory all '@opencode-ai/sdk', OpenCode process, '/api/openchamber', 'OPENCODE_*', 'OPENCHAMBER_*', OpenCode config, and OpenCode event consumers.
5. Define final PiChamber names for new routes, IPC messages, daemon commands, persisted sidecars, and environment variables. Historical MIT notices are the only intentional OpenCode references in final runtime/product code.
6. Do not ship a production engine selector or a long-lived dual-runtime UI.

The implementation may use an isolated branch/spike to establish Pi behavior, but the released product must not retain a supported OpenCode path.

Workstream 1: Build the session-runtime daemon

Create a PiChamber-owned daemon module under the web/runtime ownership boundary. The exact filenames should follow local module conventions discovered at implementation time; the conceptual owners are:

packages/web/server/lib/pi/session-daemon/

daemon lifecycle and composition
 private IPC server and protocol schema
 session registry and directory scoping
 Pi SDK factory/resource loader/model runtime integration
 queue and session-operation handlers
 event snapshot/replay publisher
 provider/auth bridge
 attachment handoff

Responsibilities:

- start one daemon per local PiChamber host;
- bind only a local Unix socket or Windows named pipe;
- reject non-PiChamber clients at the private boundary;
- construct Pi SDK sessions with selected server-side cwd and normal Pi 'agentDir';
- load Pi settings, credentials, models, sessions, skills, prompts, and 'AGENTS.md' through Pi's normal discovery rules;
- hold active session runtimes in a registry keyed by authoritative Pi session identity and cwd;
- rebind Pi event subscriptions after 'new', resume, fork, clone, or other session replacement;
- dispose idle runtime objects safely without deleting Pi session JSONL;
- expose daemon health/version/capability information to the web server;
- report malformed or unreadable session JSONL as a visible failure, never as an authoritative empty session.

The daemon should not expose an HTTP/LAN port, browser authentication surface, pairing endpoint, or user-facing route.

Workstream 2: Define the private IPC contract

Use a versioned, typed, request/response plus event protocol. The protocol is PiChamber-owned; it is not Pi RPC and it is not Pi's experimental remote protocol.

Minimum command families:

```
health
projects.list / project.select
sessions.list / create / open / delete / rename / archive metadata bridge
sessions.tree / navigate / fork / clone
sessions.prompt / steer / followUp / abort
sessions.setModel / setThinking / compact
providers.list / status / login / logout / custom-model mutation
resources.list for skills, prompts, and AGENTS.md scopes
attachments.create
```

Minimum event families:

```
session.snapshot
session.lifecycle
assistant.message.start / delta / end
assistant.thinking.delta
session.tool.start / update / end
session.queue
session.model
session.thinking
session.compaction
session.error
session.interrupted
```

Protocol requirements:

- every session event has a monotonically increasing sequence number;
- snapshots carry the last included sequence number;

- clients reconnect from snapshots, not inferred empty state;
- partial assistant deltas are assembled from Pi event lifecycle and 'contentIndex', while finalized Pi messages are authoritative;
- a daemon request failure includes a stable machine-readable code and does not mutate unrelated sessions;
- an unavailable daemon is visibly unavailable, not represented as an idle or empty session;
- no credentials, pairing secrets, bearer tokens, or user attachment bytes are logged by protocol diagnostics.

Workstream 3: Replace the OpenCode API and UI data layer

Create Pi-native service and shared contract modules, conceptually:

```
packages/ui/src/lib/pi/
  client/service facade
  API types
  event/snapshot reducer helpers
  model/provider helpers
  attachment helpers
packages/ui/src/sync/
  Pi session bootstrap, reconnect, and event reduction owners
```

Replace 'OpenCodeService' and its OpenCode SDK data shapes rather than adding a permanent emulation layer. Generic UI components may be retained, but their data access must be PiChamber/Pi-native.

The UI migration must cover:

- initial project/session bootstrap;
- authoritative session list and transcript loading;
- live message/reasoning/tool rendering;
- user prompt, steer, follow-up, and abort behavior;
- model and thinking controls;
- session name, branch/tree, fork, clone, compact, and delete flows;
- reconnect from snapshots after socket loss;
- stale runtime/directory protection when a client switches remote hosts;
- explicit interrupted/error display.

Do not preserve OpenCode endpoint names, event names, SDK client classes, or type aliases merely to reduce refactor size.

Workstream 4: Integrate Pi configuration and providers

Implement a Pi-aware settings/provider boundary in the daemon and web API.

Pi-owned reads/writes:

```
~/.pi/agent/settings.json
~/.pi/agent/auth.json
~/.pi/agent/models.json
~/.pi/agent/trust.json
<cwd>/pi/settings.json
```

PiChamber-owned settings include:

```
PiChamber default model
PiChamber default thinking level
PiChamber small model and walkthrough-model overrides
Magic prompts
PiChamber theme/layout preferences
```


Archive metadata

Remote hosts, pairing, UI authentication, relay/tunnel state

Provider work must include:

- provider/model discovery from Pi's model runtime;
- authenticated/unavailable status without secret disclosure;
- API-key write flow directly to Pi storage;
- Pi-supported OAuth, browser, device-code, and manual-code interactions adapted to the browser UI;
- custom provider/model editing backed by Pi 'models.json';
- refresh/error state with explicit timeout and stale-data behavior.

Workstream 5: Native resources, trust, and behavior settings

Implement:

- native skill discovery views;
- prompt-template-backed snippets UI;
- Magic Prompt UI retaining PiChamber ownership;
- global/project 'AGENTS.md' view/edit behavior;
- project-trust dialogs and persisted Pi trust decisions;
- extension-disabled-by-default resource loader configuration;
- extension settings placeholder/configuration only when its security contract is ready.

Do not port OpenCode catalog copying, config-entity mutation, agent editor, plugin management, or optimizer behavior.

Workstream 6: Attachments and local developer integrations

Attachments:

- add a bounded server upload route authenticated through the existing PiChamber client model;
- write original bytes to OS temporary storage with a generated Pi-style name;
- sanitize client-facing filenames and prevent path traversal;
- pass only server-local path/metadata to the Pi session;
- distinguish uploaded temporary files from server-selected '@path' references;
- make a missing temporary path an explicit attachment/tool failure;
- do not add a PiChamber sweeper, persistence store, or cleanup job in this phase.

Port independent integrations in this order:

1. server filesystem and directory picker;
2. server terminal;
3. Git status/diff/stage/commit/identity/integration flows;
4. previews;
5. walkthrough and its explicit small-model invocation;
6. GitHub workflows, notifications, quota, mini-chat, and other follow-up integrations.

Workstream 7: Remote, desktop, and deployment lifecycle

Retain the existing single-owner authentication and connection product model:

- password login, passkeys, trusted-device bearer tokens, and pairing;
- direct server URLs, LAN, reverse proxy, private network, and Tailscale-style connectivity;

- Desktop SSH forwarding;
- Cloudflare/ngrok tunnel options and E2EE relay as later port work;
- host switching and remote-instance UI.

The web server must supervise daemon lifecycle:

1. 'pichamber serve' starts or reuses one daemon for its host.
2. The server health-checks the private daemon and reconnects after server restarts.
3. 'pichamber stop' stops both the public server and its daemon.
4. Electron starts the same daemon as agent-runtime infrastructure while keeping the PiChamber backend in Electron's in-process boundary.
5. Planned daemon restart waits for idle work where practical.
6. Daemon failure interrupts active runs and triggers authoritative client resync.

Workstream 8: Update ownership

Replace update behavior with explicit ownership:

- PiChamber release upgrades update the embedded Pi SDK version.
- A separately installed Pi CLI remains independent and is never mutated by PiChamber.
- Electron keeps native application update behavior.
- Headless 'pichamber update' installs a new PiChamber package before requesting a restart and must be supervisor/deployment-aware.
- Docker and systemd deployments report/use their normal image/package deployment mechanism rather than unsafe in-app replacement.
- Remove all separate OpenCode upgrade checks, routes, toasts, binary handling, and update capability logic.

Workstream 9: Delete OpenCode ownership and update documentation

After Pi core behavior is covered by tests and runtime smoke validation, remove:

```
@opencode-ai/sdk dependency and lockfile entries
packages/ui/src/lib/opencode/
packages/web/server/lib/opencode/
managed OpenCode lifecycle/proxy/watcher/upgrade code
OpenCode binary preparation and Electron resources
OpenCode VS Code process/proxy code when the VS Code port begins
OpenCode config editors, route handlers, and tests
OpenCode-specific docs, install prerequisites, Docker mounts, scripts, and update UI
OpenCode environment variables, event names, endpoint contracts, and runtime globals
```

Update current product documentation, install documentation, deployment documentation, CLI help, package metadata, and changelog entries to describe PiChamber accurately. Preserve license notices and historical attribution required by MIT.

Relative Timeline and Phase Gates

This is a dependency-based implementation timeline, not a calendar commitment. Estimates must be revised after the SDK-daemon spike and provider OAuth investigation.

Window | Deliverable | Exit gate

Week 0 | Architecture spike | Daemon imports Pi SDK, opens/creates a Pi session with normal dotfiles, streams text/tool events over private IPC, and survives a web-server reconnect.

Weeks 1-2 | Session daemon and IPC foundation | Lifecycle ownership, health, typed IPC, project/cwd scoping, session registry, snapshot/event sequencing, queue behavior, and crash-state handling have focused tests.

Weeks 3-4 | Pi-native API and shared UI state | OpenCode client/store replacement powers session list, transcript, prompt, steer/follow-up, abort, model/thinking, tree/fork/clone, and reconnect in the web UI.

Week 5 | Pi settings, providers, resources | Pi auth/models/settings integration, PiChamber defaults, provider UI, trust, AGENTS.md, native skills/templates, Magic Prompts, and extension-disabled policy work end to end.

Week 6 | Core server workspace integrations | Filesystem, terminal, Git, temporary path attachments, direct local/VPS connection, trusted-device auth, and Electron daemon lifecycle are usable.

Week 7 | Pi core cutover | Remove released OpenCode runtime path, run regression and remote/desktop smoke tests, and update core documentation. This is the first Pi-native usable release gate.

Weeks 8-10 | Retained follow-up product features | Walkthrough/small model first, then previews, GitHub, notifications, quota, archive UI refinements, mini-chat, tunnel/relay work, and updater hardening.

Later | Intentional expansion | Pi package/skills UI, native extensions/subagents, folders/retention, VS Code, hosted mobile, Capacitor mobile, and optional modes/GUI extension API.

Validation Plan

Daemon and IPC tests

Add focused tests for:

- daemon start/reuse/stop and socket/pipe permissions;
- daemon unavailable versus empty/idle state;
- selected cwd and normal Pi agent-directory discovery;
- session create/open/list/name/tree/fork/clone/delete;
- Pi event mapping, delta assembly, finalized-message authority, snapshots, and event sequence monotonicity;
- equal-client queue ordering, steer/follow-up behavior, abort, and one-active-run invariant;
- daemon crash/planned restart/reconnect behavior;
- malformed Pi JSONL handling and no false empty bootstrap;
- resource trust decisions and extensions-disabled default;
- credential redaction and provider status behavior.

API/UI tests

Cover:

- client bootstrap/reconnect after runtime or directory switch;
- stale request/event rejection after remote-host switch;
- streaming message/tool/reasoning rendering;
- model/thinking precedence and session-persisted change behavior;
- archive metadata round-trip without modifying Pi JSONL;
- temporary attachment path handoff, missing-file error, size/path validation, and no base64 transcript expansion;
- Pi prompt-template snippets, skill discovery, Magic Prompts, and 'AGENTS.md' scope editing.

Runtime and platform smoke tests

Use disposable homes, Pi agent directories, project directories, and PiChamber data roots. Never point

tests at a developer's real credentials or sessions.

Verify:

1. A fresh web server starts/reuses the daemon and reaches a Pi SDK session without a globally installed 'pi' command.
2. Existing Pi settings/auth/models/session files are read from the configured test agent directory.
3. A browser reconnect and web-server restart leave a long-running daemon session alive.
4. Daemon termination marks active work interrupted and a reconnect restores persisted session history without claiming completion.
5. A paired remote desktop client operates only on a VPS/server project directory.
6. Provider settings never return stored secrets.
7. A file attachment becomes a temporary server path available to Pi tools rather than a base64 model-context payload.
8. Electron starts its backend in-process and reaches its private session daemon correctly.
9. Direct remote host connection, password/passkey/token auth, and pairing continue to protect the server.
10. No released runtime path requires OpenCode or a separately installed Pi CLI.

Repository checks

Use package scripts as the command source of truth. For executable changes, run focused package tests plus affected type-check/lint/build commands. For shared UI/runtime contracts, run workspace type-check/lint and affected web/Electron builds. Run 'bun run dead-code' whenever source files/exports/import shapes are added, deleted, or renamed.

This document is under repo-root 'docs/', not 'packages/docs/content/docs/'; 'bun run docs:validate' validates public MDX/sidebar content and does not cover this planning file. Validate this file with focused Markdown/link review in addition to the relevant executable checks once implementation begins.

Completion Criteria

The Pi core cutover is complete when all of the following are true:

- PiChamber serves usable web and Electron Pi sessions through its SDK-backed private daemon.
- No user needs a globally installed Pi CLI or a separately running Pi server.
- Pi normal dotfiles are honored and PiChamber never duplicates provider credentials.
- New-session model/thinking defaults follow the agreed PiChamber-first fallback order.
- Trusted paired clients can share a session with correct queueing and reconnect behavior.
- Pi session JSONL remains authoritative; daemon failure is explicit and cannot become an empty/success state.
- Provider UI, project trust, native skills/prompts, snippets, AGENTS.md behavior, temporary path attachments, files, terminal, Git, and direct remote hosting function as designed.
- The final released runtime has no OpenCode process, SDK dependency, proxy, config, updater, compatibility API, or configuration requirement.
- Required MIT attribution remains intact.
- Focused tests, affected workspace checks, dead-code inspection where applicable, and web/Electron/remote smoke validation pass without new regressions.

Appendix A: Grilling Session Decision Snapshot

This appendix is a normalized record of the design-grilling session that produced this plan. It preserves the actual decisions while replacing short answers such as "recommended" with their full meaning. Where a later answer superseded an earlier one, the final decision is stated explicitly.

A.1 Product identity and release scope

Q1 - Initial supported surfaces

PiChamber will initially support Electron desktop and the local/self-hosted web server. A PiChamber server must also be usable on a VPS and reachable by the desktop UI. VS Code, hosted mobile, and Capacitor mobile are deferred until the Pi session host and transport are proven.

Q2 / Q53 - Agent-host architecture

The original idea of an in-process SDK server was revisited after comparing OpenCode's process boundary, Pi RPC, Pi's experimental remote protocol, and the 'pi-web' project. The final answer is a dedicated PiChamber session-runtime daemon that imports Pi SDK directly. The public PiChamber server/Electron backend communicates with the daemon over private local IPC. Pi's experimental remote protocol is not used. Per-session Pi RPC workers are not the chosen production architecture.

Q3 - Breaking migration policy

PiChamber is a clean Pi-only breaking migration. It will preserve MIT notices and attribution from OpenChamber/OpenCode where legally required, but it will not retain OpenCode execution, configuration editing, session semantics, or a dual-engine compatibility mode. Existing OpenChamber/VPS data does not need migration.

Q4 / Q19 - Core product boundary and timing

The first usable Pi-native milestone is a reliable Pi core: direct local/VPS server connection, Pi sessions/transcripts, live streaming, model/thinking controls, native session history, tools, trust, and the core Git/files/terminal loop. Retained advanced features are follow-up milestones rather than blockers for Pi cutover.

Q5 / Q12 / Q18 - Extensions and subagents

PiChamber will eventually have a PiChamber-specific extension API, but it is not immediate work. Pi native third-party extensions are disabled by default in the core milestone and later become configurable under explicit trust/allow policies. Pi subagents are extension-provided task delegation rather than a Pi core primitive, so subagents wait until extension support is intentionally designed and tested.

A.2 Remote hosting, connectivity, and control

Q6 / Q13 - Connection methods

PiChamber retains every existing OpenChamber connection direction: direct local/LAN IPs, reverse proxy/HTTPS, private networks such as Tailscale, Desktop SSH forwarding, managed tunnels, and E2EE relay. Direct server connections are proved first; tunnel and relay provider ports occur after core Pi connectivity is stable.

Q7 - Ownership model

Each server is a single personal owner/trust-domain workspace server. PiChamber is not designed as a multi-tenant account platform in this phase.

Q8 - Remote workspace ownership

When connected to a VPS/server, all Pi work occurs on that server's filesystem and user account. PiChamber will not tunnel the desktop filesystem, terminal, Git credentials, or previews into the remote agent runtime.

Q14 - Authentication and pairing

PiChamber retains the existing password, passkey, trusted-device token, pairing link/QR, and connection-candidate model. No legacy OpenChamber server data migration is required. The model remains a prerequisite for safely using direct, SSH, tunnel, and relay connections.

Q20 - Shared session control

All trusted paired clients have equal control of a session. The server/daemon is authoritative for one active run and the queue. New input becomes steer or follow-up work; abort affects the shared run; every client receives the resulting state.

Q39 / Q56 - Restart behavior

Web/browser/API restart must not necessarily stop daemon-owned sessions. Planned daemon restart waits for work to settle where practical. Unexpected daemon failure marks active runs interrupted and forces rebuild/resync from persisted Pi sessions; it must never be represented as successful completion.

Q54 / Q55 - Daemon lifecycle and transport

'pichamber serve' owns starting, reusing, health-checking, and stopping one daemon per host; Electron starts the same runtime as agent infrastructure while retaining its in-process PiChamber backend rule. The daemon communicates only over Unix sockets on macOS/Linux and named pipes on Windows. Browsers never reach it directly.

A.3 Pi configuration, sessions, providers, and security

Q11 / Q17 / Q21 / Q24 - Configuration ownership and defaults

PiChamber honors Pi's existing dotfiles for credentials, models, global/project settings, sessions, resources, and trust. PiChamber stores only PiChamber-owned UI/server settings. PiChamber has its own default model, default thinking, small model, walkthrough model, and Magic Prompt settings. For a new session, configured PiChamber defaults override Pi settings; when unset, Pi settings are used. A model or thinking choice made inside an existing session remains that session's own Pi-recorded choice.

Q23 - Default agent

Pi has no ordinary normal-session "agent" selector equivalent to OpenCode's default agent. PiChamber removes the default-agent setting rather than inventing a misleading mapping. A future modes/extensions design may introduce an intentional PiChamber concept.

Q15 - Project trust

PiChamber preserves Pi project-trust semantics. Untrusted project-local resources are not silently loaded or executed. PiChamber supplies the GUI decision required by its SDK host.

Q16 - Tool approval policy

PiChamber follows Pi's default no-popup model. It does not add a mandatory interactive confirmation for every tool call in the core milestone.

Q25 / Q27 - Provider UI and secrets

The Providers UI remains. It supports Pi-native API key setup, Pi-supported OAuth/device/browser flows, provider status, model discovery, and later custom provider/model editing. Credentials are written to Pi configuration and never returned to browsers after entry. All providers Pi exposes are eligible for the UI; OpenCode-branded model providers are not specially excluded merely because the OpenCode runtime is removed.

Q51 - OpenCode-provider exception

The "no OpenCode runtime" rule does not hide ordinary upstream Pi model providers named OpenCode Zen/Go. Those are provider catalog entries, not an OpenCode server dependency.

Q30 - Scheduled tasks

Scheduled tasks and loop files are removed completely for this migration rather than deferred as a retained runtime feature.

A.4 Features retained, deferred, and removed

Q9 - Advanced workflow features

Goal Mode, Plan Mode, pinned/reinjected context, session assist, auto-review, multi-run/fusion, and managed worktrees are removed. Changes walkthrough is retained as the advanced workflow worth porting first.

Q10 - Developer workspace features

Git UI and identities, GitHub PR/issues, server terminal, filesystem browser/editor, preview proxy, notifications, quota display, Magic Prompts/snippets, themes, and desktop mini-chat remain product features. Their implementation is phased after the Pi core where indicated by the plan.

Q26 / Q28 - Small model

PiChamber retains a small/utility-model setting. After removal of goals, session assist, and voice/TTS, its surviving invocations are Changes Walkthrough, Git commit-message generation, PR-description generation, and selection-to-project-notes summaries. Calls remain explicit feature behavior, not automatic background activity added by the Pi core.

Q29 / Q33 - Skills

Pi native skill discovery is used directly. The existing OpenChamber catalog is not copied mechanically because it is a Git/ZIP copy installer tied to OpenCode discovery and locations. A later PiChamber skills UI may curate packages but must use Pi's package/resource mechanisms, scopes, updates, and extension security model.

Q34 - Snippets and Magic Prompts

Snippets become native Pi prompt templates. Magic Prompts remain PiChamber-owned feature configuration rather than generic Pi template files.

Q31 / Q35 - Session organization

Pi native session history, naming, tree navigation, fork, clone, and deletion are used. PiChamber archive is retained as shared, host-level UI metadata that hides a Pi session only from PiChamber. Custom folders, retention cleanup, sharing, and OpenCode-style revert are not part of the core migration.

Q32 / Q37 - Behavior settings

The OpenCode prompt optimizer is removed. PiChamber exposes global and selected-project Pi 'AGENTS.md'

behavior with appropriate scope/trust labeling.

A.5 Attachments, CLI, updates, and deletion

Q38 / Q42 / Q45 / Q47 / Q49 / Q52 - Attachments

The existing inline base64 attachment approach is replaced because it can bloat model context for files such as XLSX. PiChamber uploads original bytes to server-local OS temporary paths resembling Pi clipboard files, then gives Pi a path for normal tool use. Attachments are not copied into the project and are not persisted under PiChamber data. PiChamber intentionally does not implement an attachment cleanup/retention policy; OS temporary-file handling can be configured later outside the PiChamber scope.

Q40 - CLI role

PiChamber CLI is a server-management tool rather than an active agent-control interface. Retain service lifecycle, logs, startup, connection URL, update, and tunnel management. Remove OpenCode-specific session/model/project/control/schedule CLI behavior. Users who want terminal agent operation use Pi directly.

Q41 / Q43 / Q48 - Update ownership

The existing product has separate web/global-package update, Electron app update, and OpenCode upgrade mechanisms. PiChamber removes the OpenCode upgrade path. PiChamber releases own the embedded SDK version. Electron retains native app updates. Headless update is explicit and deployment/supervisor-aware: install before restart, never copy the current stop-first/no-rollback updater, and defer Docker/systemd replacement to their normal deployment workflows.

Q50 - API cutover

PiChamber creates a Pi-native server API and event contract. It may reuse generic visual components, but it replaces OpenCode service classes, OpenCode SDK data types, OpenCode event reducers, API compatibility routes, and OpenCode-shaped stores rather than preserving a long-lived facade.

Final confirmation

The user confirmed this complete direction after the session-daemon architecture, private IPC, lifecycle, provider, attachments, scope, and phase decisions were recorded. This plan is the implementation handoff for that agreed direction.